

UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO (UFERSA)
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS (CMPF)
DEPARTAMENTO DE ENGENHARIAS E TECNOLOGIA (DETEC)

SISTEMAS INTELIGENTES

Pré-Processamento de Dados

Prof. Pedro Thiago Valério de Souza
UFERSA – Campus Pau dos Ferros
pedro.souza@ufersa.edu.br

O que é Pré-Processamento de Dados?

- Conjunto de etapas que transformam dados “crus” em uma forma **adequada** para o Aprendizado de Máquina (AM).
- Costuma consumir a **maior parte** do esforço de um projeto de AM.
- Objetivos: melhorar a *qualidade* dos dados e adequá-los às exigências dos algoritmos.
- Etapas típicas:
  - Integração e seleção de dados;
  - Amostragem e balanceamento;
  - Limpeza (incompletos, inconsistentes, ruído);
  - Transformação (codificação, escala);
  - Redução de dimensionalidade.
- Princípio: *garbage in, garbage out*.

Integração de Dados

Combinar dados de **múltiplas fontes** em um conjunto único e coerente.

- Fontes: bancos de dados, planilhas, arquivos, APIs.
- Principais Desafios:
  - **Identificação de Entidades:** reconhecer que “CPF” e “documento” referem-se ao mesmo atributo (*schema matching*);
  - **Conflitos de Valor:** mesma grandeza em unidades diferentes (kg vs. g; R$ vs. US$);
  - **Redundância:** atributos repetidos ou deriváveis de outros;
  - **Duplicação** de registros da mesma entidade.
- Operação central: *junção* (*join*) por uma chave comum.

Integração de Dados

Operação de *join*: Reunir dados de fontes diferentes numa única tabela, combinando as linhas que concordam em uma **chave comum** (aqui, `id`).

Clientes

id	nome
1	Ana
2	João
3	Bia

Compras

id	produto	valor
1	livro	80
1	caneta	5
2	mochila	150
4	fone	60

A chave `id=3` (Bia) não tem compra; a compra `id=4` não tem cliente. O tipo de *join* decide o destino desses casos.

Integração de Dados

Operação de *join*:

Clientes

id	nome
1	Ana
2	João
3	Bia

Compras

id	produto	valor
1	livro	80
1	caneta	5
2	mochila	150
4	fone	60

Inner (Clientes $\bowtie$ Compras): Só chaves em *ambas*

id	nome	produto	valor
1	Ana	livro	80
1	Ana	caneta	5
2	João	mochila	150

Left (Clientes $\overset{\text{L}}{\bowtie}$ Compras): mantém todos os *Clientes*

id	nome	produto	valor
1	Ana	livro	80
1	Ana	caneta	5
2	João	mochila	150
3	Bia	`NaN`	`NaN`

Inner descarta a Bia (sem compra). **Left** mantém todos os clientes, preenchendo com `NaN` quem não casou.

Integração de Dados

Operação de *join*:

Clientes

id	nome
1	Ana
2	João
3	Bia

Compras

id	produto	valor
1	livro	80
1	caneta	5
2	mochila	150
4	fone	60

Right (Clientes $\bowtie\!\sqsubset$ Compras): preserva todas as *Compras*

id	nome	produto	valor
1	Ana	livro	80
1	Ana	caneta	5
2	João	mochila	150
4	`NaN`	fone	60

Full outer (Clientes $\sqsupset\!\bowtie\!\sqsubset$ Compras): preserva tudo de *ambas*

id	nome	produto	valor
1	Ana	livro	80
1	Ana	caneta	5
2	João	mochila	150
3	Bia	`NaN`	`NaN`
4	`NaN`	fone	60

Right mantém a compra órfã (`id=4`). **Full outer** mantém a Bia *e* a compra órfã.

Eliminação Manual de Atributos

Remoção de colunas com base em **conhecimento do domínio**, antes de qualquer método automático.

- Candidatos naturais a remoção:
  - **Identificadores** (ID, CPF, matrícula);
  - Atributos **constantes** ou quase constantes;
  - Atributos **irrelevantes** para o problema;
  - Colunas com **excesso de valores faltantes**;
  - Atributos que causam **vazamento** (*data leakage*): contêm informação do alvo que não estaria disponível na predição.
- Vantagem: reduz dimensionalidade e ruído mantendo interpretabilidade.
- Atenção: essa etapa é diferente da *seleção automática* de atributos (vista adiante).

Amostragem

- **Amostragem:** selecionar um **subconjunto** representativo dos dados.
- Motivaçoes:
  - Conjuntos muito grandes: custo de memória e tempo;
  - Prototipagem e testes rápidos;
  - Criar partições de treino/validação/teste.
- Princípio: a amostra deve **representar** bem a população (mesmas propriedades estatísticas), de preferência com a mesma distribuição estatística dos dados.
- Tipos de amostragem:
  - Amostragem aleatória simples;
  - Amostragem estratificada;
  - Amostragem progressiva.

Amostragem

Amostragem Aleatória Simples:

- Cada amostra tem a **mesma probabilidade** de ser escolhida.
- Pode ser de dois tipos: sem reposição e com reposição.
  - **Sem reposição:** cada elemento entra no máximo uma vez.
  - **Com reposição:** um elemento pode ser sorteado mais de uma vez (base do *bootstrap*).
- A amostragem com reposição tende a ser mais simples, pois a probabilidade de escolher um item não muda ao longo do tempo.
- Em situações em que o tamanho da amostra é muito menor do que o tamanho do conjunto de dados, a amostragem sem reposição e com reposição levam a resultados semelhantes.

Amostragem

Amostragem Estratificada:

- Utilizada quando a base de treinamento tem classes com propriedades diferentes, por exemplo, número de objetos diferentes.
- A população é dividida em **estratos** (subgrupos homogêneos, ex.: as classes do alvo).
- Amostra-se **dentro de cada estrato**, preservando as **proporções** originais.
- Garante que grupos minoritários apareçam na amostra.
- Utilizado quando é necessário criar a divisão entre conjunto de treinamento, validação e teste.

Exemplo: Se o alvo tem 90% de “não” e 10% de “sim”, a amostra estratificada mantém $90/10$; a aleatória simples poderia distorcer isso.

Amostragem

Amostragem Progressiva:

- Começa com uma amostra **pequena** e a **aumenta** gradualmente.
- A cada passo, treina-se o modelo e mede-se o desempenho.
- Para-se quando o desempenho **estabiliza** (curva de aprendizado atinge um platô), já que amostrar mais não compensa.
- Útil para descobrir o *tamanho de amostra suficiente* sem usar todos os dados de uma vez.

Limpeza de Dados

Objetivo: Corrigir ou remover problemas que degradam a qualidade dos dados. Os dados podem ser:

- **Incompletos:** valores faltantes.
- **Inconsistentes:** valores contraditórios ou impossíveis.
- **Redundantes:** registros duplicados ou atributos deriváveis.
- **Com ruído:** erros aleatórios / valores muito discrepantes.

Limpeza de Dados

Dados Incompletos: Valores ausentes (`NaN`, vazio, “?”).

ID	Idade	Renda	Cidade
1	25	3000	Natal
2	**?**	4500	Mossoró
3	40	**?**	**?**

Tratamentos:

- Remover linhas/colunas (se poucas ou muito incompletas);
- **Imputação**: média/mediana (numérico), moda (categórico);
- Imputação por modelo (ex.: k-NN, regressão);
- Criar indicador de “estava faltando”.

Limpeza de Dados

Dados Incompletos: Preencher faltantes, numérico pela média ou mediana; **categórico** pela moda.

Antes (com faltantes)

ID	Idade	Cidade
1	25	Natal
2	**?**	Mossoró
3	40	Natal
4	30	**?**
5	25	Natal

Depois (imputado)

ID	Idade	Cidade
1	25	Natal
2	**27,5**	Mossoró
3	40	Natal
4	30	**Natal**
5	25	Natal

Valores conhecidos de **Idade**: $\{25, 40, 30, 25\}$

$$\text{média} = \frac{25 + 40 + 30 + 25}{4} = 30, \qquad \text{mediana} = \frac{25 + 30}{2} = 27{,}5$$

Cidade: moda = Natal (3 ocorrências). Foi escolhido a *mediana* da Idade (mais robusta a *outliers* que a média) e a *moda* da Cidade.

Limpeza de Dados

Dados Inconsistentes: Valores que **violam regras** do domínio ou se contradizem.

ID	Idade	Data de nasc.	UF
1	**-5**	2000-01-01	RN
2	30	**1990-13-40**	**XX**
3	200	1998-05-10	RN

- Problemas: idade negativa, idade impossível (200), data inválida (mês 13, dia 40), UF inexistente.
- **Tratamento:** validação por faixas/regras, tabelas de domínio, checagem de coerência entre atributos.

Limpeza de Dados

Dados Redundantes: Registros duplicados ou atributos deriváveis de outros.

ID	Nome	Ano nasc.	Idade
1	Ana	1998	27
1	**Ana**	**1998**	**27**
2	João	2001	24

- Linha 2 é **duplicata** da linha 1.
- "Idade" é **derivável** de "Ano nasc." → redundância entre atributos (alta correlação).
- **Tratamento:** remover duplicatas; eliminar atributos redundantes ou altamente correlacionados.

Limpeza de Dados

Dados com Ruído: Erros aleatórios ou valores muito discrepantes (*outliers*).

Medição	Temperatura (°C)
1	22,5
2	23,1
3	**250,0**
4	22,8

O valor 250 °C é um ruído evidente.

- **Técnicas:** *binning* (suavização por faixas), regressão (ajustar a uma função), **detecção de outliers**.
- Técnicas de Detecção de *outliers*:
  - Detecção utilizando amplitude inter-quartílica;
  - Detecção utilizando LOF (*Local Outlier Factor*).

Limpeza de Dados

Detecção de *outliers* utilizando amplitude inter-quartílica: É *outlier* todo valor fora do intervalo:

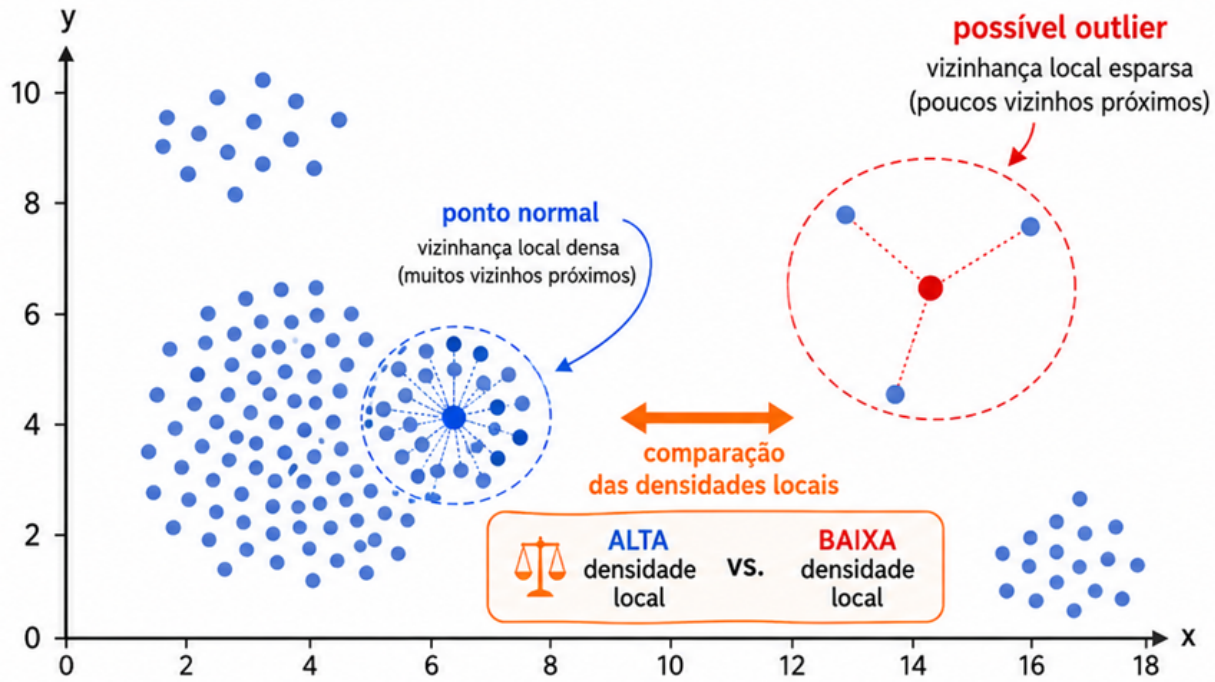
$$\left[\, Q_1 - 1{,}5\, IQR,\;\; Q_3 + 1{,}5\, IQR \,\right]$$

- **Vantagens:** simples, robusto e não assume distribuição normal.
- **Limitação:** avalia **um atributo por vez** (univariado) – não captura combinações anômalas de atributos.

Limpeza de Dados

Detecção de *outliers* por LOF (*Local Outlier Factor*): Método baseado em densidade.

- Ideia: comparar a densidade *local* de um ponto com a densidade dos seus vizinhos (k vizinhos mais próximos).
- Um ponto em região muito menos densa que a vizinhança recebe um LOF alto.
- Interpretação do escore:
  - LOF ≈ 1: densidade semelhante à dos vizinhos (normal);
  - LOF $\gg 1$: ponto isolado $\rightarrow$ provável *outlier*.
- **Vantagem:** detecta *outliers* locais, que regras globais (como o IQR) não pegam.



1) Encontre os k-vizinhos mais próximos.

2) Estime a densidade local do ponto e de seus vizinhos.

3) Compare as densidades: se o ponto tiver densidade muito menor, seu LOF será alto.

INTERPRETAÇÃO

- **LOF ≈ 1:** ponto semelhante aos vizinhos
- **LOF > 1:** ponto mais isolado
- **LOF muito alto:** forte candidato a outlier

LOF = comparação entre densidade local do ponto e dos vizinhos.

Limpeza de Dados

Detecção de *outliers* por LOF (*Local Outlier Factor*):

Algoritmo 1 Local Outlier Factor (LOF).

Entrada: conjunto $\mathbf{X}$ com n pontos; nº de vizinhos k; limiar τ
Saída: escore $\mathrm{LOF}_k(p)$ de cada ponto e os *outliers* marcados

para cada $p \in \mathbf{X}$ **faça**
- $d_k(p) \leftarrow$ distância de p ao seu k-ésimo vizinho mais próximo
- $N_k(p) \leftarrow \{\, o \in \mathbf{X} \setminus \{p\} : d(p,o) \leq d_k(p) \,\}$ `// vizinhança`

para cada $p \in \mathbf{X}$ **faça**
- **para cada** $o \in N_k(p)$ **faça**
  - $\mathrm{rd}_k(p,o) \leftarrow \max\{\, d_k(o),\, d(p,o) \,\}$ `// dist. de alcance`
- $\mathrm{lrd}_k(p) \leftarrow |N_k(p)| \,/\, \sum_{o \in N_k(p)} \mathrm{rd}_k(p,o)$ `// densidade local`

para cada $p \in \mathbf{X}$ **faça**
- $\mathrm{LOF}_k(p) \leftarrow \frac{1}{|N_k(p)|} \sum_{o \in N_k(p)} \mathrm{lrd}_k(o) \,/\, \mathrm{lrd}_k(p)$
- **se** $\mathrm{LOF}_k(p) > \tau$ **então**
  - marcar p como *outlier*

retorne escores $\mathrm{LOF}_k(\cdot)$ e os pontos marcados

Transformação de Dados

Objetivo: Adequar os atributos ao que o algoritmo espera.

- **Simbólico → Numérico:** algoritmos numéricos precisam de números.
- **Numérico → Simbólico:** discretizar em faixas.
- **Transformação de Numéricos:** colocar atributos em escalas comparáveis (normalização, padronização).

Transformação de Dados

Conversão Simbólico → Numérico:

- **Ordinal/*Label Encoding*** (quando há ordem):

Escolaridade	Código
Fundamental	0
Médio	1
Superior	2

- **One-Hot Encoding** (categorias *nominais*, sem ordem):

Cor	é_vermelho	é_azul	é_verde
vermelho	1	0	0
azul	0	1	0

- Atenção! Usar *label enconding* em dados nominais cria uma **ordem falsa**.

Transformação de Dados

Conversão Numérico → Simbólico (discretização): Agrupar valores contínuos em faixas (*bins*).

Idade	Faixa
17 – 34	jovem
35 – 69	adulto
70 –	idoso

Métodos de Agrupamento:

- **Largura igual** (*equal-width*): faixas de mesmo tamanho.
- **Frequência igual** (*equal-frequency*): mesmo nº de amostras por faixa.

É util para reduzir ruído e para algoritmos que preferem categorias.

Transformação de Dados

Transformação de Dados Numéricos: Muitos algoritmos (k-NN, SVM, redes neurais) são sensíveis à **escala** dos atributos.

- **Normalização** (*min-max*) – leva ao intervalo $[0,1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- **Padronização** (*z-score*) – deixa os dados com média 0 e desvio 1:

$$x' = \frac{x - \mu}{\sigma}$$

Redução de Dimensionalidade

- **Maldição da dimensionalidade:** com muitos atributos, os dados ficam esparsos e as distâncias perdem significado.
- Pode-se aplicar técnicas de redução de dimensionalidade para **reduzir o número de atributos** mantendo a maior parte da informação.
- Benefícios: menos custo, menos *overfitting*, melhor visualização.
- Duas abordagens:
  - **Agregação (extração):** criar novos atributos combinando os originais (ex.: PCA);
  - **Seleção:** escolher um subconjunto dos atributos originais.

Redução de Dimensionalidade

Análise de Componentes Principais (PCA):

- Um certo objeto com n atributos, pode ser entendido como um ponto em um espaço n-dimensional.
- Cada *freature* é uma dimensão do espaço.
- O PCA projeta os dados em um espaço de menor dimensão (k-dimensional), em que $k < n$, preservando a maior parte da variância dos dados.
- O novo espaço é definido por **componentes principais**, que são combinações lineares dos atributos originais.
- Assim como o espaço original, o novo espaço é ortogonal, ou seja, as componentes principais são independentes entre si.

Redução de Dimensionalidade

Análise de Componentes Principais (PCA):

Algoritmo 2 Análise de Componentes Principais (PCA).

Entrada: matriz $\mathbf{X}_{n \times d}$; número de componentes k
Saída: dados projetados $\mathbf{Z}_{n \times k}$

$\bar{\mathbf{x}} \leftarrow$ vetor média $\mathbf{X}$ (média calculada nas colunas)
$\tilde{\mathbf{X}} \leftarrow \mathbf{X} - \mathbf{1}\,\bar{\mathbf{x}}^\top$ // centralizar os dados
$\boldsymbol{\Sigma} \leftarrow \frac{1}{n-1}\,\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ // covariância
Obter λ_j e $\mathbf{v}_j$ tais que $\boldsymbol{\Sigma}\,\mathbf{v}_j = \lambda_j\,\mathbf{v}_j$
Ordenar $\mathbf{v}_j$ por λ_j decrescente
$\mathbf{W}_k \leftarrow [\,\mathbf{v}_1\ \mathbf{v}_2\ \cdots\ \mathbf{v}_k\,]$ // k maiores autovalores
$\mathbf{Z} \leftarrow \tilde{\mathbf{X}}\,\mathbf{W}_k$
retorne $\mathbf{Z}$

- $\mathbf{1}$ é um vetor-coluna de n elementos iguais a 1.
- Variância explicada pela componente j: $\lambda_j / \sum_l \lambda_l$.

Redução de Dimensionalidade

Seleção de Atributos: Manter um **subconjunto** dos atributos originais (preserva interpretabilidade). Podem ser classificados em três tipos:

- **Filtros (*filter*):** avaliam cada atributo por uma medida estatística, independente do modelo.
  - Exemplos de medida: correlação, informação mútua, teste χ^2, variância.
- **Empacotadores (*wrapper*):** usam o **desempenho de um modelo** para avaliar subconjuntos (mais caro, mais preciso).
  - **Seleção para frente** (*forward*): Começa vazio e **adiciona** o atributo que mais melhora o modelo, um a um.
  - **Eliminação para trás** (*backward*): Começa com todos e **remove** o menos útil a cada passo.
  - **Stepwise:** A seleção parte de um ponto, e combina adição e remoção de atributos.
  - **Busca exaustiva:** Testa todos os subconjuntos. É ótima, porém 2^d combinações (inviável para d grande).
- **Embutidos (*embedded*):** a seleção ocorre **durante** o treino.
  - Exemplo: Regularização Lasso (ℓ_1), importância de atributos em árvores.

Desbalanceamento de Classes

- Uma ou mais classes têm **muito menos** exemplos que as demais.
- Ex.: detecção de fraude (99% legítimas, 1% fraudes).
- Consequências:
  - O modelo tende a favorecer a classe majoritária;
  - A **acurácia engana**: prever sempre "majoritária" dá 99% de acurácia e 0% de detecção de fraude.
- Use métricas adequadas: **precisão**, **revocação**, **F1**, **AUC-ROC** e a matriz de confusão.

Desbalanceamento de Classes

Técnicas para Dados Desbalanceados:

- **Nível dos dados (reamostragem):**
  - *Oversampling* da minoritária: duplicação aleatória, SMOTE, ADASYN;
  - *Undersampling* da majoritária: remoção aleatória, *Tomek links*, ENN.
- **Nível do algoritmo:**
  - **Pesos de classe** (penalizar mais o erro na minoritária);
  - Aprendizado sensível a custo (*cost-sensitive*).
- ***Ensembles* balanceados:** Balanced Random Forest, EasyEnsemble.
- **Métricas** apropriadas em vez da acurácia (acima).

Desbalanceamento de Classes

SMOTE (*Synthetic Minority Over-sampling Technique*): Cria exemplos **sintéticos** da classe minoritária – em vez de apenas duplicar, *interpola* entre vizinhos.

1. Para cada amostra minoritária x_i, encontre seus k vizinhos mais próximos (dentro da classe minoritária).
2. Sorteie aleatoriamente um desses vizinhos, x_{zi}.
3. Gere uma amostra sintética **no segmento** entre x_i e x_{zi}:

$$x_{\text{novo}} = x_i + \lambda\,(x_{zi} - x_i), \qquad \lambda \sim U(0,1)$$

4. Repita até atingir o balanceamento desejado.

Desbalanceamento de Classes

SMOTE (*Synthetic Minority Over-sampling Technique*):

Algoritmo 3 Synthetic Minority Over-sampling Technique (SMOTE).

Entrada: conjunto minoritário $\mathbf{X}_{\min}$ com m amostras; nº de vizinhos k; nº de amostras sintéticas N

Saída: conjunto sintético $\mathbf{S}$ com N amostras

$\mathbf{S} \leftarrow \emptyset$

para cada $\mathbf{x}_i \in \mathbf{X}_{\min}$ **faça**

  $N_k(\mathbf{x}_i) \leftarrow k$ vizinhos mais próximos de $\mathbf{x}_i$ **em** $\mathbf{X}_{\min}$ `// só a minoritária`

enquanto $|\mathbf{S}| < N$ **faça**

  $\mathbf{x}_i \leftarrow$ amostra sorteada de $\mathbf{X}_{\min}$

  $\mathbf{x}_{zi} \leftarrow$ vizinho sorteado de $N_k(\mathbf{x}_i)$

  $\lambda \leftarrow$ valor sorteado de $U(0,1)$

  $\mathbf{x}_{\text{novo}} \leftarrow \mathbf{x}_i + \lambda\,(\mathbf{x}_{zi} - \mathbf{x}_i)$ `// ponto no segmento`

  $\mathbf{S} \leftarrow \mathbf{S} \cup \{\mathbf{x}_{\text{novo}}\}$

retorne $\mathbf{S}$

Desbalanceamento de Classes

SMOTE (*Synthetic Minority Over-sampling Technique*):

- Vantagem sobre a duplicação simples: aumenta a **diversidade** e reduz o *overfitting* na minoritária.
- Cuidado: pode gerar exemplos em regiões de **sobreposição** entre classes, criando ruído.
- **Aplicar apenas no treino**, nunca no conjunto de teste.
- Variantes:
  - **Borderline-SMOTE:** foca nas amostras próximas à fronteira;
  - **ADASYN:** gera mais exemplos onde a classe é mais difícil;
  - **SMOTE + Tomek** / **SMOTE + ENN:** combinam com limpeza depois.